

ДАТА ПОСТУПЛЕНИЯ (дата регистрации) оригиналов документов заявки	(21) РЕГИСТРАЦИОННЫЙ №	ВХОДЯЩИЙ №
(85) ДАТА ПЕРЕВОДА международной заявки на национальную фазу		
☐ (86) (регистрационный номер международной заявки и дата международной подачи, установленные получающим ведомством) ☐ (87) (номер и дата международной публикации международной заявки)	АДРЕС ДЛЯ ПЕРЕПИСКИ (почтовый адрес, фамилия и инициалы или наименование адресата) Сафронов Павел Андреевич, 196602, г. Санкт-Петербург, г. Пушкин, ул. Камероновская, д. 10, стр. 1, кв. 13. Телефон: 8 (911) 837-97-62 Факс: Электронная почта: safronius@yandex.ru	
ЗАЯВЛЕНИЕ о выдаче патента на полезную модель	В Федеральную службу по интеллектуальной собственности Бережковская наб., 30, корп. 1, Москва, Г-59, ГСП-3, 125993, Российская Федерация	
(54) НАЗВАНИЕ ПОЛЕЗНОЙ МОДЕЛИ Цифровой двойник грузового автомобиля с применением распределения реестров для контроля, диагностики и оптимизации эксплуатации (К-16)		
(71) ЗАЯВИТЕЛЬ (фамилия, имя, отчество (последнее – при наличии) физического лица или наименование юридического лица (согласно учредительным документам), место жительства или место нахождения, название страны и почтовый индекс) Сафронов Павел Андреевич Российская Федерация (RU), 196602, г. Санкт-Петербург, г. Пушкин, ул. Камероновская, д. 10, стр. 1, кв. 13.		ИДЕНТИФИКАТОРЫ ЗАЯВИТЕЛЯ ОГРН КПП (при наличии) ИНН (при наличии) 663002003060 СНИЛС 130-606-929 32 ДОКУМЕНТ (вид, серия, номер) ПАСПОРТ 7503 296688 КОД СТРАНЫ (если он установлен) 07778627
Телефон: 8 (911) 837-97-62 Факс: E-mail: safronius@yandex.ru		
☐ полезная модель создана за счет средств федерального бюджета Заявитель является: ☐ государственным заказчиком ☐ муниципальным заказчиком исполнитель работ _____ (указать наименование) ☐ исполнителем работ по: ☐ государственному контракту ☐ муниципальному контракту ☐ соглашению о предоставлении субсидии ☐ гранту ☐ государственному заданию ☐ инициативному заданию заказчик работ _____ (указать наименование) Контракт (соглашение, договор) от _____ № _____ В соответствии с условиями государственного контракта (договора, соглашения) права на созданные результаты интеллектуальной деятельности принадлежат: ☐ Российской Федерации, от имени которой выступает государственный заказчик ☐ Российской Федерации, от имени которой выступает государственный заказчик, и исполнителю совместно ☐ исполнителю ☐ иное		
(74) ПРЕДСТАВИТЕЛЬ (ПРЕДСТАВИТЕЛИ) ЗАЯВИТЕЛЯ		☐ патентный поверенный

☐ представитель по доверенности ☐ представитель по закону		
(указываются фамилия, имя, отчество (последнее – при наличии) лица, назначенного заявителем своим представителем для ведения дел по получению патента от его имени в Федеральной службе по интеллектуальной собственности или являющегося таковым в силу закона)		
Фамилия, имя, отчество (последнее – при наличии)	Телефон: Факс: E-mail:	
Адрес		
Срок представительства (если к заявлению приложена доверенность представителя заявителя, срок может не указываться)	Регистрационный номер патентного поверенного _____	
(72) Автор (фамилия, имя, отчество (последнее – при наличии))	Адрес места жительства, включающий официальное наименование страны и ее код по стандарту ВОИС ST. 3 (если он установлен)	
Сафронов Павел Андреевич	196602, г. Санкт-Петербург, г. Пушкин, ул. Камероновская, д. 10, стр. 1, кв. 13.	
Овчинников Сергей Викторович	188662, Ленинградская область, Всеволожский район, г. Мурино, бульвар Менделеева д.11 к.4, кв.67	
Дубровских Денис Сергеевич	195252, г. Санкт-Петербург, ул. Северный проспект д. 87 к.4, кв.133	
☐ Я (мы) _____ (фамилия, имя, отчество (последнее – при наличии))		
Прошу (просим) не упоминать меня (нас) как автора (авторов) при публикации сведений о выдаче патента		
Подпись (подписи) автора (авторов)		
☐ Просьба автора (авторов) не упоминать его (их) при публикации прилагается (отмечается при подаче заявки в электронном виде)		
ПЕРЕЧЕНЬ ПРИЛАГАЕМЫХ ДОКУМЕНТОВ	Количество листов в экз.	Количество экз.
☐ описание полезной модели	13	2
☐ формула полезной модели	1	2
☐ чертеж (чертежи) и иные материалы, ☐ в том числе трехмерная модель полезной модели в электронной форме фигуры чертежей, предлагаемые для публикации с рефератом _____ (указать)	18	2
☐ реферат	1	2
☐ копия документа, подтверждающего уплату пошлины (пошлин) (представляется по собственной инициативе заявителя)	1	1
☐ ходатайство о предоставлении права на освобождение от уплаты патентных пошлин или их уплату в уменьшенном размере	1	1
☐ копия первой заявки (при испрашивании конвенционного приоритета)		

☐ перевод заявки на русский язык		
☐ доверенность		
☐ просьба автора (авторов) не упоминать его (их) при публикации		
☐ другой документ (указать наименование документа)		
☐ дополнительные листы к настоящему заявлению	3	1
☐ копия документов заявки (описание, формула полезной модели, чертежи (если имеются) и реферат) на машиночитаемом носителе <u>CD-R</u> (указать вид носителя)	1	1
Подтверждаю, что копия документов заявки на машиночитаемом носителе является точной копией документов, представленных на бумажном носителе.		

ЗАЯВЛЕНИЕ НА ПРИОРИТЕТ (заполняется только при испрашивании приоритета более раннего, чем дата подачи заявки)

Прошу установить приоритет полезной модели по дате

- 1 ☐ подачи первой заявки в государстве – участнике Парижской конвенции по охране промышленной собственности (пункт 1 статьи 1382 Гражданского кодекса Российской Федерации (Собрание законодательства Российской Федерации, 2006, № 52, ст. 5496; 2014, № 11, ст. 1100) (далее – Кодекс)
- 2 ☐ поступления дополнительных материалов к более ранней заявке (пункт 2 статьи 1381 Кодекса)
- 3 ☐ подачи более ранней заявки (пункт 3 статьи 1381 Кодекса)
- 4 ☐ подачи (приоритета) первоначальной заявки (пункт 4 статьи 1381 Кодекса), из которой выделена настоящая заявка

№ заявки	Дата испрашиваемого приоритета на основании указанной заявки	Код страны подачи по стандарту ВОИС ST.3 (если он установлен) (при испрашивании конвенционного приоритета)

ХОДАТАЙСТВО ЗАЯВИТЕЛЯ

Прошу

- ☐ начать рассмотрение международной заявки ранее установленного срока (пункт 1 статьи 1396 Кодекса)
- ☐ выдать патент на полезную модель на бумажном носителе (пункт 1 статьи 1393 Кодекса)

☐ Уплачена пошлина ☐ по подпункту _____ приложения № ☐ 1 ☐ 2 к Положению о патентных и иных пошлинах за совершение юридически значимых действий, связанных с патентом на изобретение, полезную модель, промышленный образец, с государственной регистрацией товарного знака и знака обслуживания, с государственной регистрацией и предоставлением исключительного права на географическое указание, наименование места происхождения товара, а также с государственной регистрацией отчуждения исключительного права на результат интеллектуальной деятельности или средство индивидуализации, залога исключительного права, предоставления права использования такого результата или такого средства по договору, перехода исключительного права на такой результат или такое средство без договора, утвержденному постановлением Правительства Российской Федерации от 10 декабря 2008 г. № 941 (Собрание законодательства Российской Федерации, 2008, № 51, ст. 6170; 2020, № 42, ст. 6639) (далее – Положение о пошлинах)

☐ по подпункту _____ приложения № ☐ 1 ☐ 2 к Положению о пошлинах

Сведения о плательщике (указывается фамилия, имя, отчество (последнее – при наличии) или наименование юридического лица)

Идентификаторы плательщика, указываемые в документе, подтверждающем уплату пошлины:

☐ Для российского юридического лица:
ИНН:
КПП:
☐ Для российского физического лица:
ИНН: 594703280840
СНИЛС: 147-725-289-92
вид, серия и номер документа,
удостоверяющего личность плательщика:
ПАСПОРТ 57 13 № 051789

☐ Для иностранного юридического лица:
(идентификаторы указываются в одном из двух сочетаний)
☐ КИО (если имеется): ☐ ИНН или его аналог
в иностранном государстве
(если имеется):
КПП (если имеется): КПП (если имеется):
☐ Для иностранного физического лица:
ИНН или его аналог в иностранном государстве:
СНИЛС (если имеется):
вид, серия и номер документа,
удостоверяющего личность плательщика:
☐ Гражданство (по общероссийскому классификатору стран мира): ☐ Без гражданства

Заявителю известно, что в соответствии с пунктом 4 части 1 статьи 6 Федерального закона от 27 июля 2006 г. № 152-ФЗ «О персональных данных» (Собрание законодательства Российской Федерации, 2006, № 31, ст. 3451; 2020, № 17, ст. 2701) (далее – Федеральный закон «О персональных данных») Федеральная служба по интеллектуальной собственности осуществляет обработку персональных данных субъектов персональных данных, указанных в заявлении, в целях и объеме, необходимых для предоставления государственной услуги. Заявитель подтверждает наличие согласия других субъектов персональных данных, указанных в заявлении (за исключением согласия представителя), на обработку их персональных данных, приведенных в настоящем заявлении, в Федеральной службе по интеллектуальной собственности в связи с предоставлением государственной услуги. Согласия оформлены в соответствии со статьей 9 Федерального закона «О персональных данных».

(заполняется только заявителями по российским заявкам)

Заявителю известно, что с информацией о состоянии делопроизводства по заявлению, в том числе о направленных заявителю документах, можно ознакомиться на официальных сайтах Федеральной службы по интеллектуальной собственности (www.rupto.ru) и Федерального государственного бюджетного учреждения «Федеральный институт промышленной собственности» (www.fips.ru) в информационно-телекоммуникационной сети «Интернет».

Заявитель подтверждает достоверность информации, приведенной в настоящем заявлении.

Подпись

П.А. Сафронов

Подпись, фамилия, имя, отчество (последнее – при наличии) заявителя или представителя заявителя, или иного уполномоченного лица, дата подписи (при подписании от имени юридического лица подпись руководителя или иного уполномоченного на это лица удостоверяется печатью при ее наличии).

Реферат

Цифровой двойник грузового автомобиля обеспечивает контроль, диагностику и повышение эффективности эксплуатации грузового транспорта.

Цифровой двойник грузового автомобиля с блокчейн - интеграцией для контроля, диагностики и оптимизации эксплуатации (К-16) включает аппаратную часть (датчики, ESP32, 5G, АТЕСС608) и программную (AI-алгоритмы, блокчейн, ERP-интеграцию).

Преимущества:

Повышает эффективность безопасности, снижения затрат, защиты данных.

Описание полезной модели

1. Название полезной модели

Цифровой двойник грузового автомобиля с применением распределения реестров для контроля, диагностики и оптимизации эксплуатации (К-16)

2. Область техники

Полезная модель относится к области цифровых технологий, Интернета вещей (IoT), искусственного интеллекта (AI) и блокчейн-систем.

Цифровой двойник грузового автомобиля предназначена для контроля данных узлов и агрегатов грузового автомобиля и прогнозирования их износа.

3. Уровень техники

Существующие решения применяют системы телематики контролирующие базовые параметры грузового автомобиля (скорость, расход топлива). Недостатки существующих решений:

3.1. Отсутствие криптографической защиты данных (применение аппаратных модулей безопасности АТЕСС608).

3.2. Отсутствие:

3.2.1. Контроля прогнозирования износа шаровых опор, тормозной системы, нагрузки на оси.

3.2.2. Оптимизации маршрутов на основе AI в реальном времени.

4. Сущности полезной модели

Цифровой двойник грузового автомобиля включает:

4.1. Аппаратную часть:

- Датчики контроля параметров (двигатель, тормозная система, нагрузка на оси, GPS).
- Микроконтроллер (ESP32) с поддержкой CAN-шины для сбора данных.
- Модуль 5G передачи данных с низкой задержкой.
- Аппаратный криптомодуль (АТЕСС608) шифрования данных.
- Источник питания с подзарядкой от бортовой сети автомобиля.

4.2. Программную часть:

4.2.1. Алгоритмы машинного обучения (Isolation Forest) выявления аномалий в режиме реального времени.

4.2.2. Облачную платформу (Flask-сервер) с функциями:

- Оптимизации маршрутов – применение OpenStreetMap.
- Формирования отчетов с визуализацией нагрузок на оси грузового автомобиля в реальном времени.

4.2.3. Проведение операций с данными в блокчейн (Hyperledger Fabric) обеспечение их неизменности.

4.2.4. Удаленный доступ через мобильное приложение (React Native) и веб-клиент (React).

4.2.5. REST API применение с ERP/CRM-системами (например, Odoo).

5. Описание промышленной применимости:

Устройство применимо в логистических компаниях, автопарках и транспортных предприятиях.

Позволяет:

- Снизить расход топлива на 15%.
- Увеличить срок службы автомобилей.
- Обеспечить безопасность данных.

Формула полезной модели:

Цифровой двойник грузового автомобиля, содержащий:
датчики для контроля параметров автомобиля,
микроконтроллер ESP32 с поддержкой CAN-шины,
модуль 5G для передачи данных,
источник питания,

отличающийся тем, что дополнительно включает:

аппаратный криптомодуль АТЕСС608 для шифрования данных,
интеграцию с блокчейн-платформой через Hyperledger Fabric,
REST API для взаимодействия с ERP/CRM-системами,
алгоритмы машинного обучения для прогнозирования износа критических узлов.

1. Устройство по п. 1, отличающееся использованием алгоритма Isolation Forest для обнаружения аномалий в данных датчиков.

2. Устройство по п. 1, отличающееся функцией оптимизации маршрутов на основе данных OpenStreetMap и AI-анализа расхода топлива.

Схема
подключения ESP32 к датчикам и CAN-шине
для цифрового двойника грузовика

Основные компоненты:

1. Микроконтроллер ESP32 (основной узел сбора данных и управления).

2. CAN-интерфейс:

- MCP2515 (CAN-контроллер, подключается через SPI).

- SN65HVD230 (CAN-трансивер, преобразует сигналы TTL в CAN-уровни).

3. Датчики:

- Аналоговые: датчик температуры двигателя, датчик давления масла, датчик уровня топлива.

- Цифровые: GPS-модуль (UART), акселерометр/гироскоп (I2C, например, MPU6050).

4. Внешнее питание (при необходимости для датчиков 5В).

5. Подтягивающие резисторы (для I2C и CAN-шины).

Распиновка и подключение:

5.1. CAN-шина:

MCP2515 (подключение к ESP32 через SPI):

SCK → GPIO 18 (SPI CLK).

MISO → GPIO 19 (SPI MISO).

MOSI → GPIO 23 (SPI MOSI).

CS → GPIO 5 (выбор CAN-контроллера).

SN65HVD230 (CAN-трансивер):

CAN H и CAN L → к CAN-шине грузовика.

Питание: 3.3В или 5В (в зависимости от модели).

5.2. Аналоговые датчики (подключение к ADC ESP32):

Датчик температуры двигателя → GPIO 34 (ADC1_CH6).

Датчик давления масла → GPIO 35 (ADC1_CH7).

Датчик уровня топлива → GPIO 36 (ADC1_CH0).

5.3. Цифровые датчики:

- **GPS-модуль (UART):**

TX модуля → GPIO 16 (U2_RX).

RX модуля → GPIO 17 (U2_TX).

- **Акселерометр MPU6050 (I2C):**

SDA → GPIO 21 (I2C SDA).

SCL → GPIO 22 (I2C SCL).

5.4. Дополнительно:

Питание 3.3В датчиков и ESP32.

Подтягивающие резисторы (10 кОм) линий I2C (SDA, SCL).

Конденсатор 0.1 мкФ обеспечивающий фильтрацию помех от электростатических зарядов.

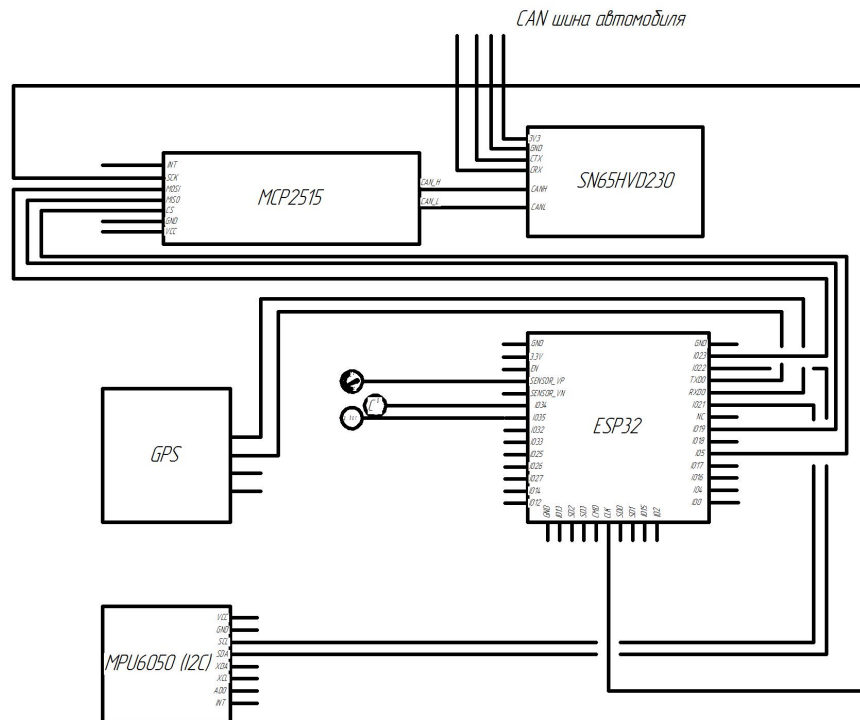


Рисунок 1 - Схема подключения ESP32 к датчикам и CAN-шине.

Примечания:

Работа CAN осуществляется на базе библиотеки ACAN ESP32 или mcp2515. Калибровка аналоговых датчиков производится ADC.

**Архитектура
системы цифрового двойника грузовика
(устройство → сервер → блокчейн → клиенты)**

Уровни системы:

- 1. Устройство (Edge Layer)**
- 2. Сервер (Cloud/Backend Layer)**
- 3. Блокчейн (Decentralized Ledger)**
- 4. Клиенты (Application Layer)**

1. Устройство (ESP32 + датчики)

Предназначение: Сбор данных с датчиков грузовика (температура, давление, GPS, ускорение и т.д.).

Технологии:

CAN-шина для связи с системами грузовика.

Wi-Fi/4G для передачи данных на сервер.

Протоколы: MQTT/HTTP отправки данных.

Пример данных

```
{  
  "truck_id": "TRUCK_001",  
  "timestamp": 1630000000,  
  "temperature": 85,  
  "fuel_level": 45,  
  "coordinates": "55.7522, 37.6156"  
}
```

2. Сервер (Cloud/Backend)

Предназначение: Прием, обработка и хранение данных.

Компоненты:

API Gateway: Прием данных с устройств (REST/MQTT).

База данных: Хранение необработанных данных (MongoDB, PostgreSQL).

Аналитика: ML-модели для прогноза износа, аномалий.

Брокер сообщений: RabbitMQ/Kafka для асинхронной обработки.

Применение с блокчейном

Основные показатели данных автомобиля (пробег, ремонты) хешируются и отправляются в блокчейн.

3. Блокчейн

Предназначение: Обеспечение неизменности и прозрачности операций с данными.

Технологии:

Распределенный реестр Hyperledger Fabric, Ethereum.

Смарт-контракты автоматизации логистических процессов:

- Фиксация пробега.
- История обслуживания.
- Проверка подлинности запасных частей.

Пример записи в блокчейн:

solidity

```
event MaintenanceRecord(  
    string truckId,  
    uint256 timestamp,  
    string action,  
    bytes32 dataHash  
);
```

4. Архитектура цифрового двойника (участники и взаимодействующие элементы и компоненты)

Предназначение: Визуализация данных и управление.

Типы клиентов:

Веб-интерфейс: Динамические графики, карта GPS, статус грузовика.

Мобильное приложение: Уведомления о испытываемых нагрузках осями.

API для сторонних систем: Применение с логистическими платформами.

Доступ к блокчейну:

Участники проверяют фактические данные через смарт-контракты (например, через Web3.js).

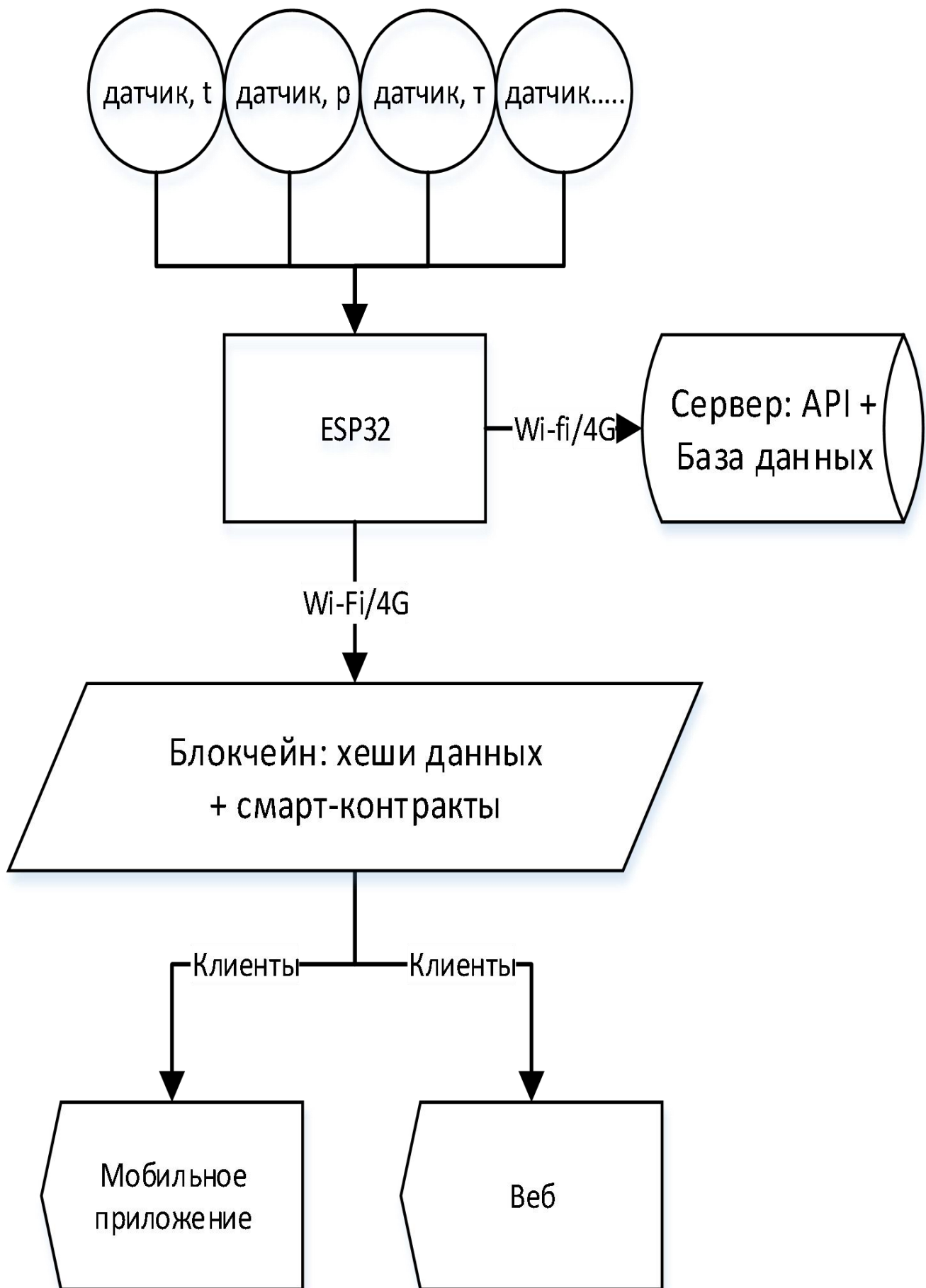


Рисунок 2 - Схема потока данных

Передача данных:

Безопасность.

Передача данных осуществляется путем их шифрования (TLS-передача, AES-хранения).

С проверкой элементов и архитектуры (JWT/OAuth2).

Масштабируемость. Микросервисная архитектура на сервере обеспечивает хранение в облачных базах данных (AWS, Azure).

Использование облачных решений (AWS, Azure).

Контроль параметров данных в распределенном реестре:

Изменения показателей данных.

Прозрачность операций с данными.

Применение:

1. Грузовой автомобиль передает данные о перегреве двигателя.
2. Сервер запускает ML-модель.
3. Производится обработка и анализ данных; фиксируется событие: «Требуется срочный ремонт».
4. Участник (менеджер автопарка) получает уведомление в реальном времени.

Пример интерфейса мобильного приложения для контроля нагрузки на оси грузовика

1. Главный экран (Дашборд)

Верхняя панель

Название грузовика (например, «КАМАЗ-65117 государственный регистрационный знак А 777АА 777»).

Текущий статус: «В движении» «На стоянке» (цветовая индикация: зелёный/серый).

Визуальный индикатор - для предупреждений о перегрузке.

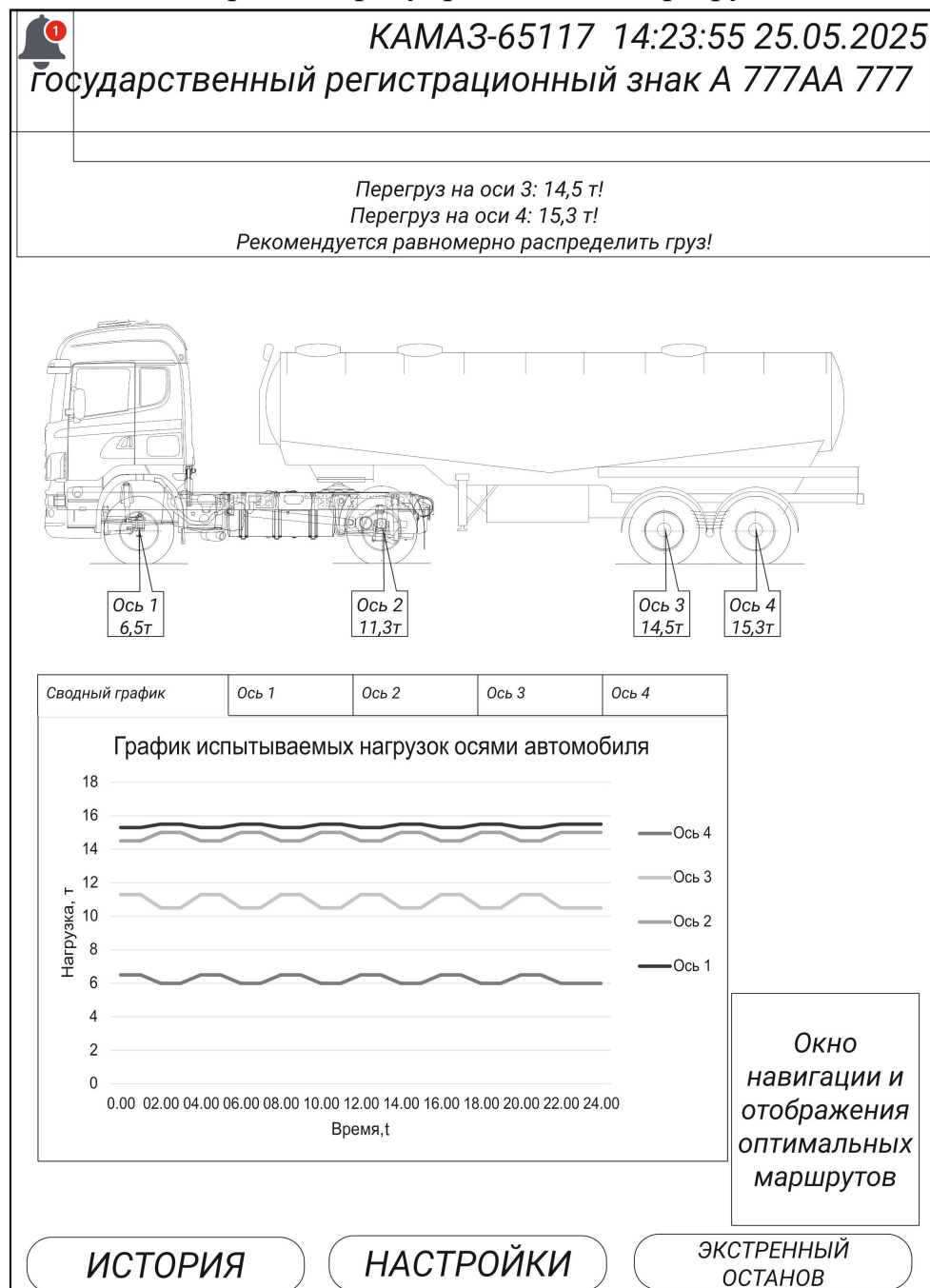


Рисунок 2 – Главный экран

Упрощённая графическая модель грузовика с обозначением осей (1-я, 2-я, 3-я ось, 4-я ось).

Индикация нагрузки на каждую ось в реальном времени

Цветовая маркировка:

Зелёный: нагрузка в норме (например, ≤ 10 тонн).

Жёлтый: приближение к пределу (10–12 тонн).

Красный: перегруз (>12 тонн).

Числовое значение нагрузки (например, "8.5 т / 10.0 т").

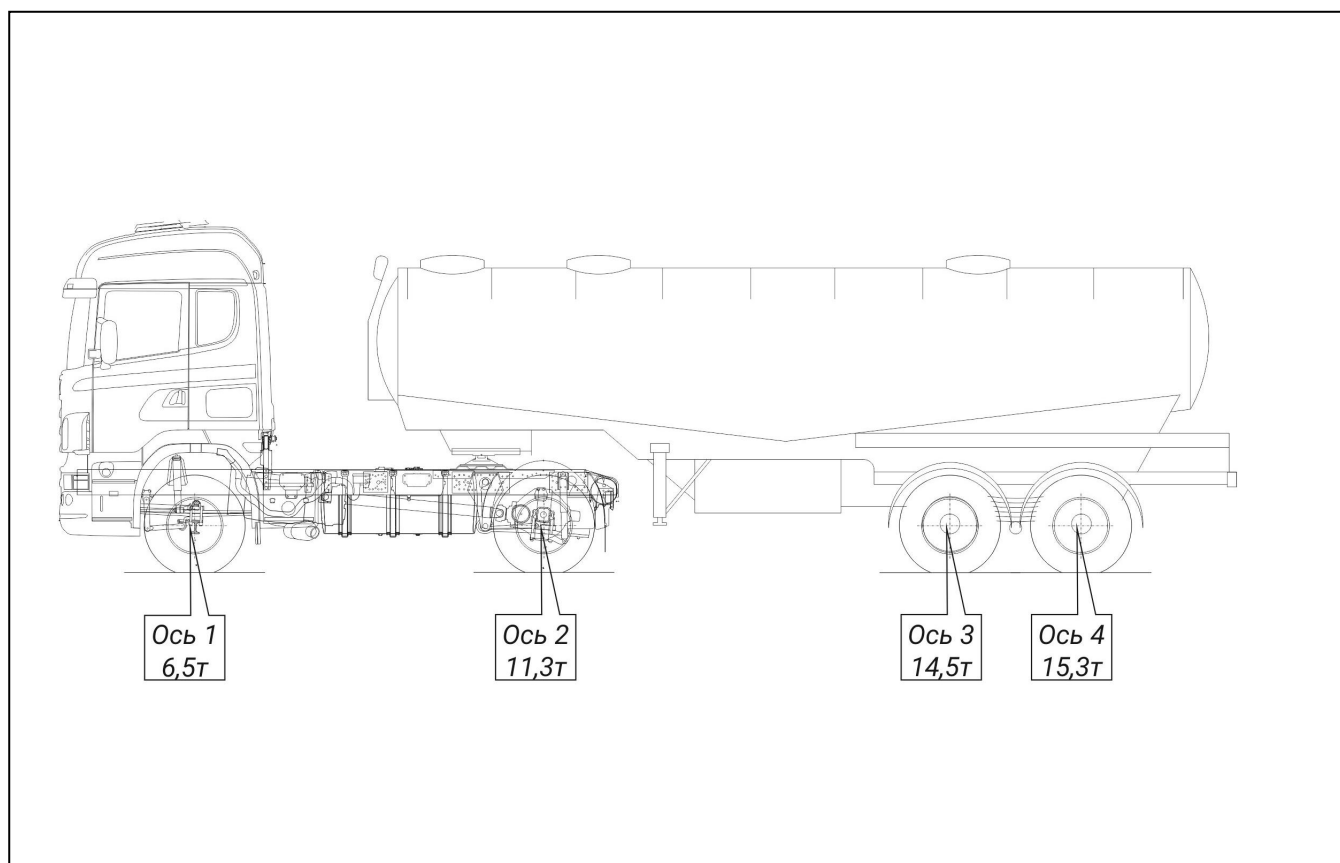


Рисунок 3 – Индикация нагрузки на каждую ось

2. Графики нагрузки

Вкладки:

Сводный график: представляет общую информацию испытываемых нагрузок по каждой оси грузового автомобиля в реальном времени;

Ось 1, Ось 2, Ось 3, Ось 4: отражает информацию о нагрузках за выбранную ось автомобиля.

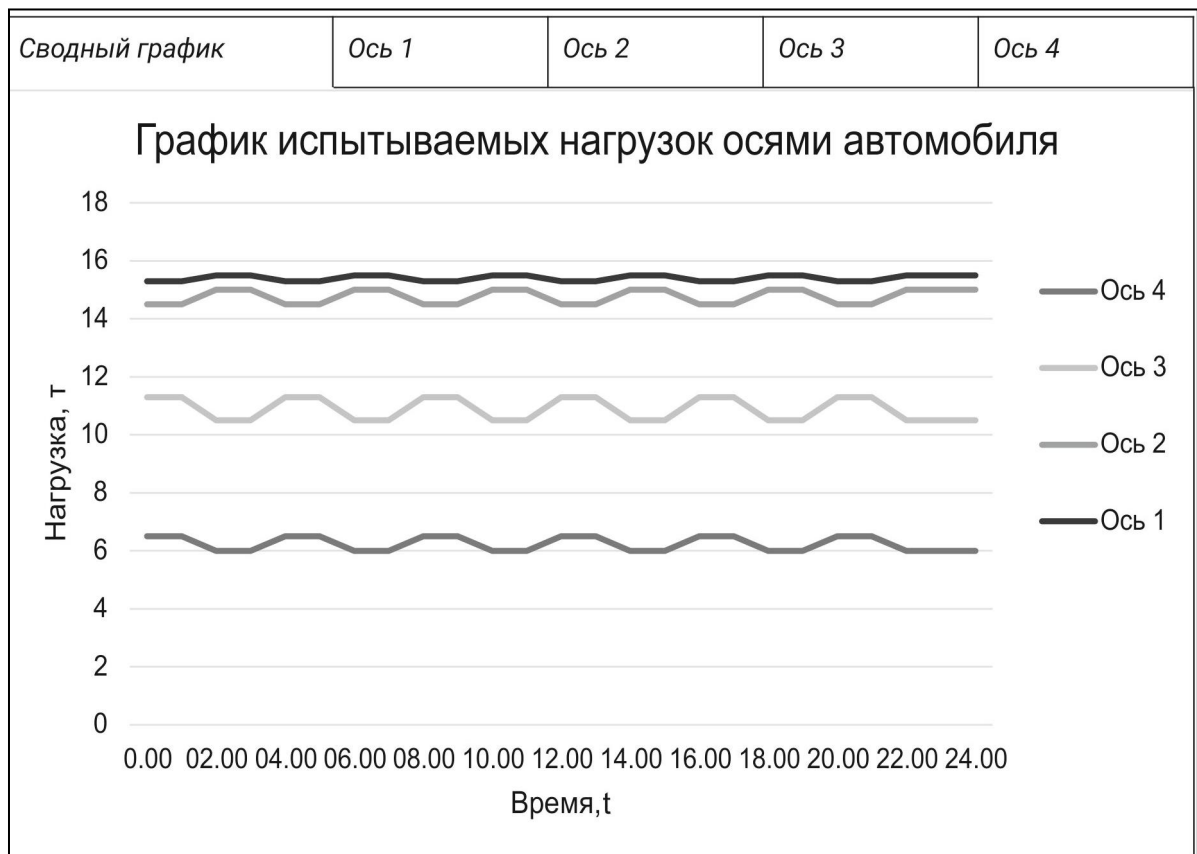


Рисунок 4 – Графики нагрузки на оси

3. Уведомления и предупреждения

Пуш - уведомления:

«Перегруз на оси 2: 12.3 т!»

«Резкое увеличение нагрузки при торможении».

Срочные действия:

Кнопка «**Экстренный останов**» (отправка команды водителю).

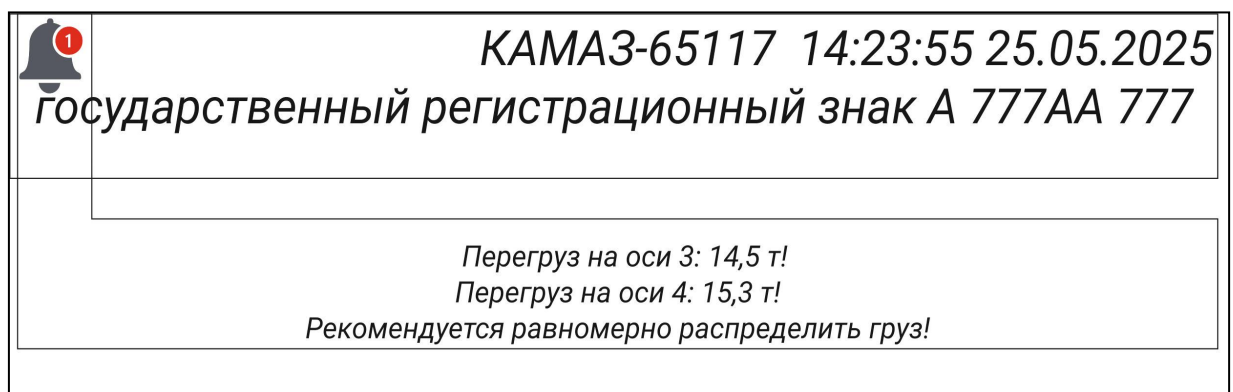


Рисунок 5 - Схема макета экрана

Технические решения

Фреймворки: React Native, Flutter.

Библиотеки графиков: Victory Native, Charts (MPAndroidChart/iOS Charts).

Применение с бэкендом: WebSocket для реального времени, REST API для истории.

Дизайн-пример:

1. Цветовая схема:

Основной: синий/серый (корпоративный стиль).

Акценты: зелёный/жёлтый/красный для индикации нагрузки.

2. Шрифты: Roboto (читаемость на маленьких экранах).

3. Анимации: Плавное обновление графиков, эффект нажатия на оси.

Визуализация:

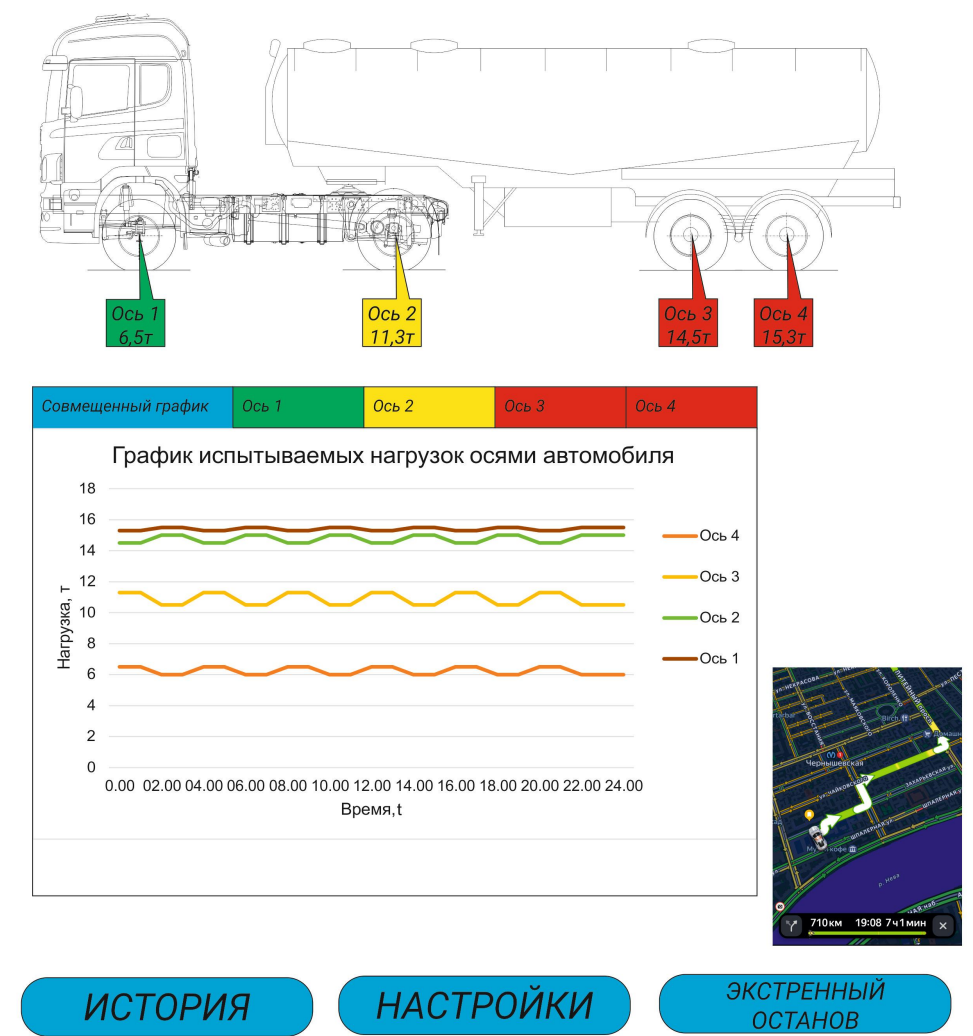
Применяйте **3D-модель грузовика** (если приложение премиум-класса) с анимацией нагрузки.

Добавьте «**геолокацию**» на карте (взаимосвязь с GPS-данными).

Перегруз на оси 3: 14,5 т!

Перегруз на оси 4: 15,3 т!

Рекомендуется равномерно распределить груз!



ИСТОРИЯ

НАСТРОЙКИ

ЭКСТРЕННЫЙ
ОСТАНОВ

Рисунок 6 - Схема макета экрана

Исходный код прошивки ESP32 (C++), сервера (Python/Flask), клиентов (React/React Native).

1. Прошивка ESP32 (C++)

```
#include <Quectel_RM500Q.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <CAN.h>
#include <mbedtls/gcm.h>
#include <ATECCX08.h>
```

```
Quectel_RM500Q rm500q(Serial2);
Preferences secureStorage;
WiFiClientSecure espClient;
PubSubClient client(espClient);
mbedtls_gcm_context gcm_ctx;
ATECCX08 atecc;
```

```
const char* mqttServer = "mqtt.thingsboard.cloud";
const int mqttPort = 8883;
```

Загрузка ключа шифрования из АТЕСС608

```
void loadAESKey(uint8_t* key) {
    if (!atecc.begin()) {
        Serial.println("Ошибка инициализации АТЕСС608");
        return;
    }
    atecc.readSlot(0, key, 32);
}
```

Инициализация GCM

```
void initAES() {
    mbedtls_gcm_init(&gcm_ctx);
    uint8_t key[32];
    loadAESKey(key);
    mbedtls_gcm_setkey(&gcm_ctx, MBEDTLS_CIPHER_ID_AES, key, 256);
}
```

Шифрование данных

```
void encryptData(uint8_t* plaintext, size_t len, uint8_t*
ciphertext, uint8_t* tag) {
    uint8_t iv[12];
    for (int i = 0; i < 12; i++) iv[i] = random(0, 255);
    mbedtls_gcm_crypt_and_tag(&gcm_ctx, MBEDTLS_GCM_ENCRYPT, len,
iv, 12, NULL, 0, plaintext, ciphertext, 16, tag);
```

```

}

void setup() {
    // Настройка 5G
    if (!rm500q.init(5)) {
        Serial.println("Ошибка инициализации RM500Q");
    }
}

Инициализация CAN
CAN.begin(500E3);

Настройка MQTT
secureStorage.begin("credentials", false);
const char* ssid = secureStorage.getString("ssid", "").c_str();
const char* password = secureStorage.getString("pass",
"").c_str();
espClient.setCACert("/spiffs/rootCA.pem");
espClient.setCertificate("/spiffs/client.crt");
espClient.setPrivateKey("/spiffs/client.key");
client.setServer(mqttServer, mqttPort);
initAES();
}

Чтение параметров
float readFuelConsumption() { .. } // Реализация из исходного кода
float readAxleLoad() { .. } // Новые датчики из 1.docx
bool readBrakeStatus() { .. } // Новые датчики из 1.docx

void sendData() {
    StaticJsonDocument<512> doc;
    doc["rpm"] = readCANData();
    doc["fuel"] = readFuelConsumption();
    doc["axle_load"] = readAxleLoad(); // Добавлено
    doc["brake_status"] = readBrakeStatus(); // Добавлено

    uint8_t ciphertext[512], tag[16];
    encryptData((uint8_t*)doc.as<String>().c_str(),
doc.as<String>().length(), ciphertext, tag);
    client.publish("sensors/truck1", (const char*)ciphertext);
}

```

2. Серверная часть (Python/Flask)

python

```
from flask import Flask, jsonify, request
from flask_jwt_extended import JWTManager, jwt_required
from pydantic import BaseModel, ValidationError
from sklearn.ensemble import IsolationForest
import numpy as np
import requests
import os
import logging
from hfc.fabric import Client
from odoo_integration import OdooERP # Интеграция из 1.docx
```

```
app = Flask(__name__)
app.config['JWT_SECRET_KEY'] = os.getenv('JWT_SECRET_KEY')
jwt = JWTManager(app)
fuel_model = IsolationForest()
fuel_data = []
```

Конфигурация ERP (из 1.docx)

```
ERP = OdooERP(
    base_url="https://erp.example.com",
    db="truck_db",
    username="admin",
    password="securepass"
)
```

Валидация данных для блокчейна

```
class BlockchainData(BaseModel):
    truck_id: str
    timestamp: str
```

Оптимизация маршрута

```
class RouteOptimizer:
    def optimize_route(self, origin, destination):
        if len(fuel_data) >= 50:
            fuel_model.fit(np.array(fuel_data).reshape(-1,1))
            url =
f"https://routing.openstreetmap.de/routed-car/route/v1/driving/{origin};{destination}?overview=full"
            response = requests.get(url).json()
            optimized_route = response['routes'][0]
            optimized_route['fuel_efficiency'] = np.mean(fuel_data) * 0.9
            return optimized_route
```

Запись в блокчейн

```

def write_to_blockchain(data):
    client = Client(net_profile="network.json")
    org1_admin = client.get_user('org1.example.com', 'Admin')
    response = client.chaincode_invoke(
        requestor=org1_admin,
        channel_name='mychannel',
        peers=['peer0.org1.example.com'],
        cc_name='truck_cc',
        fcn='createRecord',
        args=[data['truck_id'], data['timestamp']]
    )
    return response

```

Маршруты

```

@app.route('/api/erp/route', methods=['POST']) # Из 1.docx
@jwt_required()
def send_route_to_erp():
    data = request.json
    response = ERP.send_route_to_erp(data)
    return jsonify(response)

@app.route('/report', methods=['GET'])
@jwt_required()
def generate_report():
    data = {
        "fuel_usage": np.mean(fuel_data),
        "anomalies": fuel_model.predict(fuel_data).tolist(),
        "routes": RouteOptimizer().optimize_route("55.7558,37.6173",
"59.9343,30.3351")
    }
    return jsonify(data)

@app.route('/blockchain', methods=['POST'])
@jwt_required()
def blockchain_write():
    try:
        data = BlockchainData(**request.json).dict()
        write_to_blockchain(data)
        return jsonify({"status": "success"})
    except ValidationError as e:
        logging.error(f"Validation error: {e}")
        return jsonify({"error": str(e)}), 400

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, ssl_context=('fullchain.pem', 'privkey.pem'))

```

3. Клиенты

React-клиент (веб)

javascript

```
import React, { useState } from 'react';
import { PDFDownloadLink } from '@react-pdf/renderer';
import { LineChart, Line } from 'recharts';
import { useCookies } from 'react-cookie';

function ReportGenerator() {
  const [report, setReport] = useState(null);
  const [cookies] = useCookies(['crm_token', 'csrf_token']);

  const fetchReport = async () => {
    const response = await fetch('/api/report', {
      headers: { 'Authorization': `Bearer ${cookies.crm_token}` }
    });
    setReport(await response.json());
  };

  return (
    <div>
      <button onClick={fetchReport}>Сгенерировать отчет</button>
      {report && (
        <div>
          <h3>Расход топлива: {report.fuel_usage} л/100км</h3>
          <LineChart data={report.fuel_data}>
            <Line type="monotone" dataKey="fuel" stroke="#ff7300"
              />
          </LineChart>
          <PDFDownloadLink document={<ReportPDF data={report} />}
            fileName="report.pdf">
              {(loading) => loading ? 'Генерация..' : 'Скачать
PDF'}
            </PDFDownloadLink>
          </div>
        </div>
      )}
    </div>
  );
}
```

Мобильное приложение (React Native)

javascript

```
import React, { useState, useEffect } from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';
import { LineChart } from 'react-native-chart-kit';
```



```

const App = () => {
  const [report, setReport] = useState(null);
  const [status, setStatus] = useState('disconnected');

  const fetchReport = async () => {
    const response = await
fetch('https://api.example.com/report');
    setReport(await response.json());
  };

  return (
    <View style={styles.container}>
      <Button title="Обновить данные" onPress={fetchReport} />
      {report && (
        <>
          <Text>Расход топлива: {report.fuel_usage}
л/100км</Text>
          <LineChart
            data={report.fuel_data}
            width={300}
            height={200}
            chartConfig={{ color: '#ff7300' }}
          />
        </>
      )}
      <Text>Статус: {status}</Text>
    </View>
  );
};

const styles = StyleSheet.create({
  container: { padding: 20 }
});

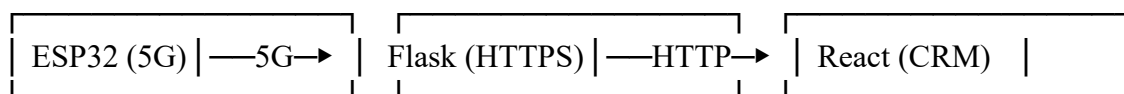
export default App;

```

4. Документация (README.md)

markdown

Архитектура



Настройка

1. ESP32:

- Загрузите сертификаты в SPIFFS.
- Используйте АТЕСС608 для хранения ключей шифрования.

2. Сервер:

```
```bash
 unicorn --certfile=fullchain.pem --keyfile=privkey.pem
app:app
```

### 3. Клиенты:

Веб: `npm install recharts @react-pdf/renderer.`

Мобильный: `npx react-native run-android.`

### 4. Интеграция с ERP/CRM

Используйте `/api/erp/route` для отправки маршрутов в Odoo.

#### Примечания:

- Все зависимости («requests», «flask-jwt-extended», «react-native-chart-kit» и др.) должны быть установлены.
- Для ESP32 требуется библиотека «mbedtls» и поддержка CAN-шины.
- Сертификаты MQTT и ключи должны быть предварительно сгенерированы.

## Документация по API для применения с Odoo ERP

### 1. Базовый URL

<https://api.example.com/erp> (замените на актуальный URL вашего сервера).

### 2. Аутентификация

Для доступа к API используется JWT (JSON Web Token).

Токен передается в заголовке Authorization:

Authorization: Bearer <ваш\_JWT\_токен>

Получить токен можно через эндпоинт аутентификации Odoo (не описан в текущей реализации; требуется настройка Odoo).

### 3. Эндпоинты

#### 3.1. Отправка данных маршрута в Odoo

Метод: POST /api/erp/route

Описание: Передает данные оптимизированного маршрута в систему Odoo ERP.

Тело запроса (JSON):

json

```
{
 "truck_id": "TRUCK-001",
 "route": {
 "origin": "55.7558,37.6173",
 "destination": "59.9343,30.3351",
 "distance_km": 650,
 "estimated_fuel_consumption": 95.5,
 "optimized_waypoints": ["55.7558,37.6173", "56.3434,38.2121",
"..."]
 },
 "timestamp": "2023-10-25T14:30:00Z"
}
```

#### Обязательные поля:

Truck id - идентификатор грузовика.

route.origin - координаты начальной точки (широта, долгота).

route.destination- координаты конечной точки.

Timestamp - время создания маршрута в формате ISO 8601.

### Ответы:

#### Успех (200 OK):

json

```
{
 "status": "success",
 "odoo_record_id": 12345,
 "message": "Маршрут успешно сохранен в Odoo."
}
```

#### Ошибка (400 Bad Request):

json

```
{
 "error": "Некорректные данные: поле 'destination' обязательно."
}
```

#### Ошибка аутентификации (401 Unauthorized):

json

```
{
 "error": "Требуется аутентификация."
}
```

## 4. Интеграция с классом OdooERP

Класс OdooERP настраивается через параметры:

python

```
ERP = OdooERP(
 base_url="https://erp.example.com", # URL Odoo
 db="truck_db", # Название базы данных
 username="admin", # Логин
 password="securepass" # Пароль
)
```

## 5. Пример использования (Python)

python

```
import requests

url = "https://api.example.com/api/erp/route"
headers = {
 "Authorization": "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
 "Content-Type": "application/json"
}

data = {
 "truck_id": "TRUCK-001",
 "route": {
 "origin": "55.7558,37.6173",
 "destination": "59.9343,30.3351",
 "distance_km": 650
 },
}
```

```
 "timestamp": "2023-10-25T14:30:00Z"
}

response = requests.post(url, json=data, headers=headers)
print(response.json())
```

## **6. Ограничения**

Лимит запросов: 100 запросов/мин на один аккаунт.

Максимальный размер тела запроса: 1 МБ.

## **7. Дополнительные настройки**

**Таймаут:** 10 секунд на выполнение запроса.

**Повторные попытки:** Рекомендуется реализовать механизм повторных запросов при кодах 5xx.

## **8. Поддержка**

Для вопросов и ошибок обращайтесь:

Email: [support@example.com](mailto:support@example.com)

Документация Odoo: <https://www.odoo.com/documentation>

Примечания:

Для тестирования используйте Postman-коллекцию (прилагается отдельно).

Все данные передаются в зашифрованном виде через HTTPS.

Обновления API публикуются в CHANGELOG.md.

## Конфигурационные файлы для развертывания блокчейн-сети Hyperledger Fabric

### 1.crypto-config.yaml

yaml

#### Определение организаций и узлов

```
OrdererOrgs:
 - Name: OrdererOrg
 Domain: example.com
 Specs:
 - Hostname: orderer

PeerOrgs:
 - Name: Org1
 Domain: org1.example.com
 EnableNodeOUs: true
 Template:
 Count: 1 # Один пир для примера
 Users:
 Count: 1 # Администратор
```

### 2. configtx.yaml

yaml

```
Organizations:
 - &OrdererOrg
 Name: OrdererOrg
 ID: OrdererMSP
 MSPDir: crypto-config/ordererOrganizations/example.com/msp
 Policies:
 Readers:
 Type: Signature
 Rule: "OR('OrdererMSP.member')"
 Writers:
 Type: Signature
 Rule: "OR('OrdererMSP.member')"
 Admins:
 Type: Signature
 Rule: "OR('OrdererMSP.admin')"

 - &Org1
 Name: Org1MSP
 ID: Org1MSP
 MSPDir: crypto-config/peerOrganizations/org1.example.com/msp
 Policies:
 Readers:
 Type: Signature
 Rule: "OR('Org1MSP.admin', 'Org1MSP.peer', 'Org1MSP.client')"
 Writers:
```

```

 Type: Signature
 Rule: "OR('Org1MSP.admin', 'Org1MSP.client')"
 Admins:
 Type: Signature
 Rule: "OR('Org1MSP.admin')"
 AnchorPeers:
 - Host: peer0.org1.example.com
 Port: 7051

Capabilities:
 Channel: &ChannelCapabilities
 V2_0: true
 Orderer: &OrdererCapabilities
 V2_0: true
 Application: &ApplicationCapabilities
 V2_0: true

Orderer: &OrdererDefaults
 OrdererType: etcdraft
 Addresses:
 - orderer.example.com:7050
 BatchTimeout: 2s
 BatchSize:
 MaxMessageCount: 10
 AbsoluteMaxBytes: 99 MB
 PreferredMaxBytes: 512 KB
 Organizations: []
 Policies:
 Readers:
 Type: ImplicitMeta
 Rule: "ANY Readers"
 Writers:
 Type: ImplicitMeta
 Rule: "ANY Writers"
 Admins:
 Type: ImplicitMeta
 Rule: "MAJORITY Admins"

Application: &ApplicationDefaults
 Organizations: []
 Policies:
 Readers:
 Type: ImplicitMeta
 Rule: "ANY Readers"
 Writers:
 Type: ImplicitMeta
 Rule: "ANY Writers"
 Admins:
 Type: ImplicitMeta
 Rule: "MAJORITY Admins"

Profiles:
 TruckNetwork:
 <<: *ChannelDefaults

```

```

Consortium: SampleConsortium
Orderer:
 <<: *OrdererDefaults
 Organizations:
 - *OrdererOrg
Application:
 <<: *ApplicationDefaults
 Organizations:
 - *Org1

```

### 3. docker-compose.yaml

```

yaml
version: "3.7"

```

```

services:
 ca.org1.example.com:
 image: hyperledger/fabric-ca:2.4
 environment:
 - FABRIC_CA_HOME=/etc/hyperledger/fabric-ca-server
 - FABRIC_CA_SERVER_CA_NAME=ca-org1
 - FABRIC_CA_SERVER_PORT=7054
 ports:
 - "7054:7054"
 command: sh -c "fabric-ca-server start -b admin:adminpw"
 volumes:

```

```

- ./crypto-config/peerOrganizations/org1.example.com/ca/:/etc/hyperle
dger/fabric-ca-server
 networks:
 - fabric

```

```

 orderer.example.com:
 image: hyperledger/fabric-orderer:2.4
 environment:
 - ORDERER_GENERAL_LISTENADDRESS=0.0.0.0
 - ORDERER_GENERAL_LISTENPORT=7050
 - ORDERER_GENERAL_LOCALMSPID=OrdererMSP
 - ORDERER_GENERAL_LOCALMSPDIR=/var/hyperledger/orderer/msp
 - ORDERER_GENERAL_TLS_ENABLED=true
 -

```

```

ORDERER_GENERAL_TLS_PRIVATEKEY=/var/hyperledger/orderer/tls/server.k
ey
 -

```

```

ORDERER_GENERAL_TLS_CERTIFICATE=/var/hyperledger/orderer/tls/server.
crt

```

```

 ports:
 - "7050:7050"
 volumes:

```

```

- ./crypto-config/ordererOrganizations/example.com/orderers/orderer.e
xample.com/:/var/hyperledger/orderer
 networks:
 - fabric

```



```

 peer0.org1.example.com:
 image: hyperledger/fabric-peer:2.4
 environment:
 - CORE_PEER_ID=peer0.org1.example.com
 - CORE_PEER_ADDRESS=peer0.org1.example.com:7051
 - CORE_PEER_LISTENADDRESS=0.0.0.0:7051
 - CORE_PEER_CHAINCODEADDRESS=peer0.org1.example.com:7052
 -
CORE_PEER_GOSSIP_EXTERNALENDPOINT=peer0.org1.example.com:7051
 - CORE_PEER_LOCALMSPID=Org1MSP
 - CORE_PEER_MSPCONFIGPATH=/etc/hyperledger/peer/msp
 - CORE_PEER_TLS_ENABLED=true
 -
CORE_PEER_TLS_CERT_FILE=/etc/hyperledger/peer/tls/server.crt
 -
CORE_PEER_TLS_KEY_FILE=/etc/hyperledger/peer/tls/server.key
 ports:
 - "7051:7051"
 volumes:

- ./crypto-config/peerOrganizations/org1.example.com/peers/peer0.org1.
example.com/:/etc/hyperledger/peer
 depends_on:
 - orderer.example.com
 networks:
 - fabric

networks:
 fabric:
 driver: bridge

```

#### 4. Скрипты для запуска сети

##### **generate-crypto.sh**

bash

```

#!/bin/bash
cryptogen generate --config=./crypto-config.yaml
--output="crypto-config"

```

##### **start-network.sh**

bash

```

#!/bin/bash
docker-compose -f docker-compose.yaml up -d

```

##### **stop-network.sh**

bash

```

#!/bin/bash
docker-compose -f docker-compose.yaml down

```

#### 5. Интеграция с Flask-сервером

Для взаимодействия с блокчейном добавьте в код сервера (Python/Flask) следующие настройки:

```
python
```

### Пример использования SDK Hyperledger Fabric

```
from hfc.fabric import Client
```

```
client = Client(net_profile="network.json")
```

```
org1_admin = client.get_user('org1.example.com', 'Admin')
```

### Вызов chaincode для записи данных

```
response = client.chaincode_invoke(
 requestor=org1_admin,
 channel_name='mychannel',
 peers=['peer0.org1.example.com'],
 cc_name='truck_cc',
 fcn='createRecord',
 args=['truck1', '2023-10-05T12:00:00Z']
)
```

### Примечания:

#### 1. Генерация сертификатов

Запустите «generate - crypto.sh» для создания криптографических материалов.

Убедитесь, что папка «crypto-config» создана корректно.

#### 2. Настройка канала

Используйте «configtxgen» для генерации «genesis-блока» и конфигурации канала.

#### 3. Развертывание сети

Запустите «start-network.sh» для развертывания контейнеров.

Убедитесь, что все узлы доступны через порты, указанные в «docker-compose.yaml».

#### 4. Интеграция

Добавьте файл «network.json» с описанием сети (пиры, orderers, каналы) для работы SDK.

